



University of Asia Pacific

Department of Computer Science & Engineering

Course Title : Microprocessors and Microcontrollers Lab

Course Code : CSE 316

Report No : 02

Report Title : Distance Measurement Using Ultrasonic
Sensor

Date : 26 - 04 - 2026

Submitted By :

Name : Md. Osman Goni

Reg. No : 23101012

Section : A1

Submitted To :

Khadiza Sultana Sayma

Lecturer, Department of CSE

University Of Asia Pacific.

1. Experiment Name

Distance measurement using ultrasonic sensor

2. Objective

To measure the distance of an object using an ultrasonic sensor (HC-SR04) and indicate the distance range using **three LEDs (green, yellow, red)** while also controlling a **servo motor angle** based on the measured distance.

3. Apparatus / Hardware & Software Requirements

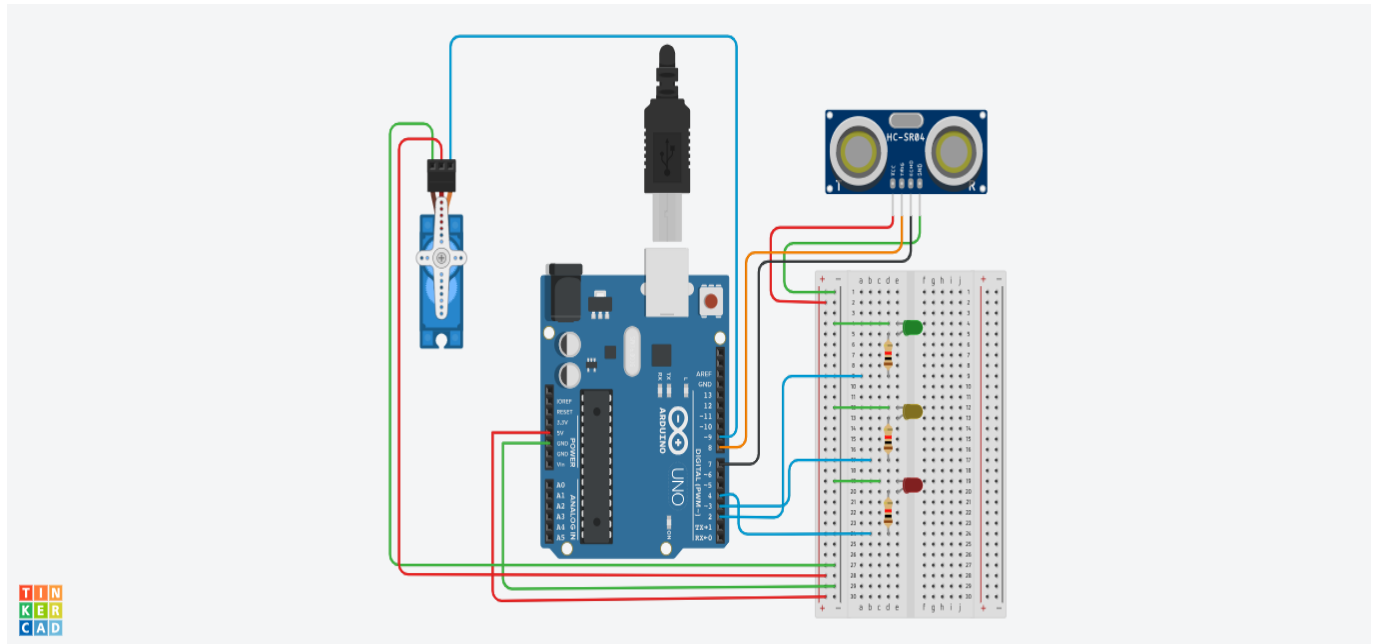
- Hardware

- Arduino Uno
- Ultrasonic sensor: **HC-SR04**
- Servo motor
- 3 LEDs: **Green, Yellow, Red**
- 3 × current-limiting resistors
- Breadboard
- Jumper wires
- USB cable / 5V supply

Software

- Arduino IDE
- Arduino Servo library (`#include <Servo.h>`) (built-in with Arduino IDE)
- Serial Monitor (inside Arduino IDE) for observing distance readings

4. Circuit Diagram / Schematic



5. Code / Assembly Program

```
#include <Servo.h>
Servo myServo;
// Pin assignments
const int trigPin = 8;
const int echoPin = 7;
const int greenLED = 2;
const int yellowLED = 3;
const int redLED = 4;
```

```
long duration;
int distance;

void setup() {
  pinMode(trigPin,
  OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(greenLED,
  OUTPUT);
  pinMode(yellowLED,
  OUTPUT);
  pinMode(redLED,
  OUTPUT);

  myServo.attach(9);
  Serial.begin(9600);
}
void loop() {
  // Send ultrasonic pulse
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
```

```
// Measure echo time
duration =
pulseIn(echoPin, HIGH);
// Calculate distance in cm
distance = duration * 0.034
/ 2;
```

```
Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");
```

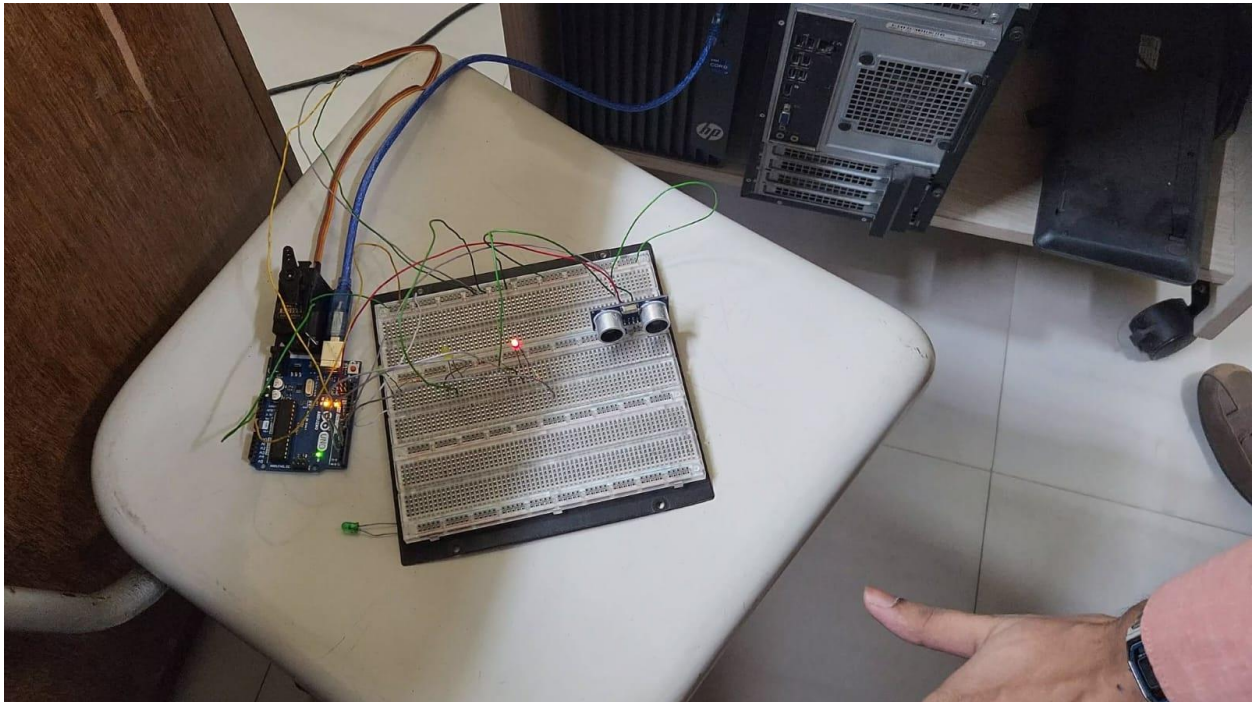
```
// Control LEDs and Servo
based on distance
if (distance <= 10) {
//Close range
    digitalWrite(greenLED,
HIGH);
    digitalWrite(yellowLED,
LOW);
    digitalWrite(redLED,
LOW);
    myServo.write(90);
}
else if (distance <= 20) {
```

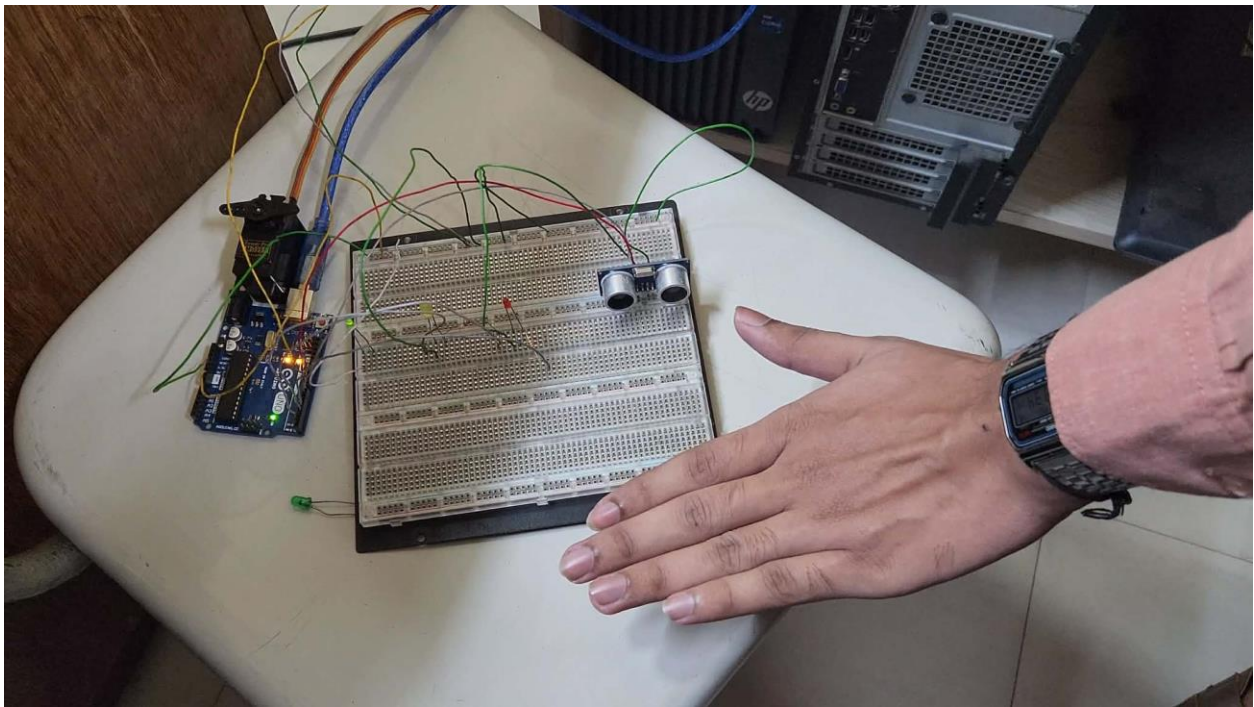
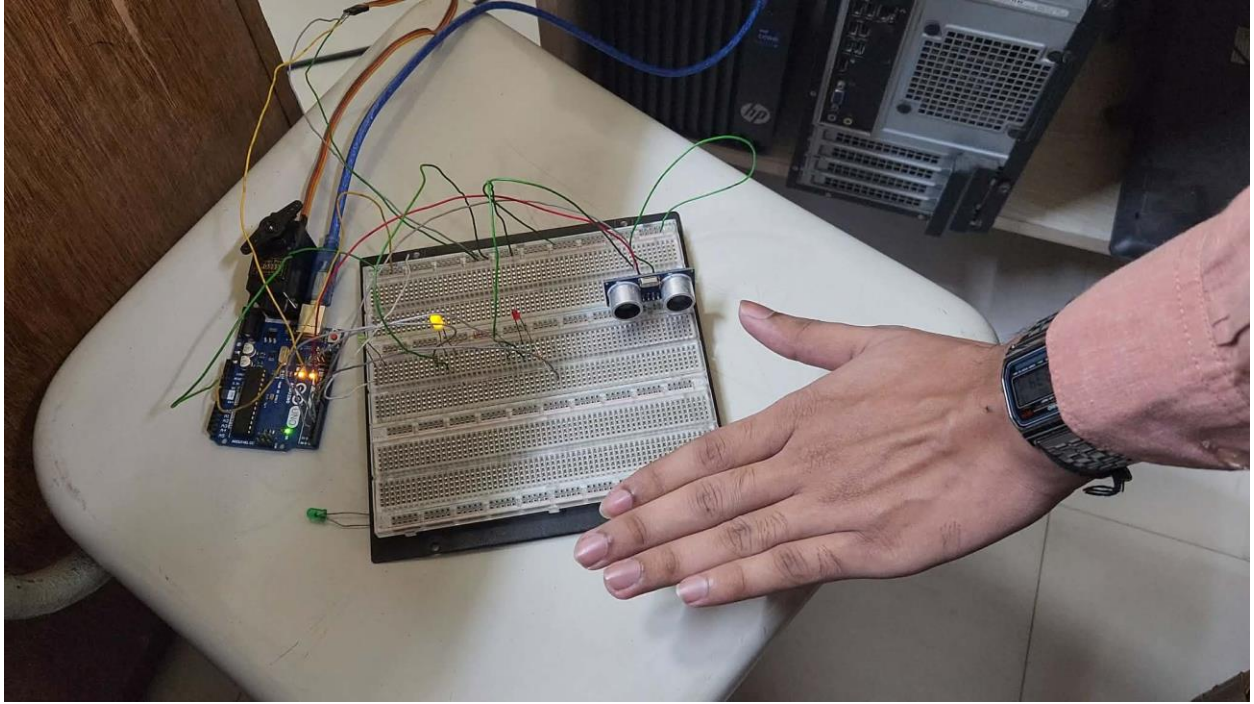
```
// Medium range
    digitalWrite(greenLED,
LOW);
    digitalWrite(yellowLED,
HIGH);
    digitalWrite(redLED,
LOW);
    myServo.write(45);
}
else {
// Far range
    digitalWrite(greenLED,
LOW);
    digitalWrite(yellowLED,
LOW);
    digitalWrite(redLED,
HIGH);
    myServo.write(0);
}

    delay(200); // Short delay
for stability
}
```

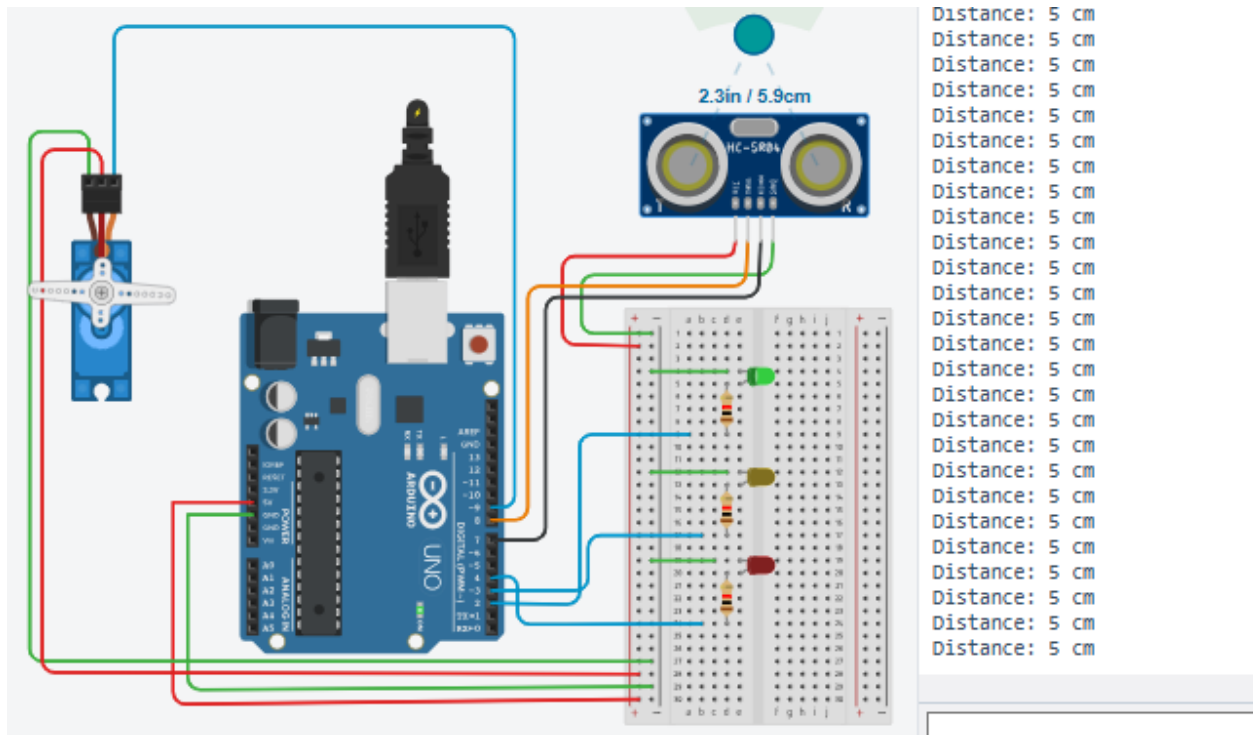
6. Output / Observations

- The Arduino sends a $10\mu\text{s}$ pulse to the **Trig** pin.
- HC-SR04 returns an echo pulse whose width depends on distance.
- Arduino calculates distance using:
distance (cm) = duration $\times$ 0.034 / 2
- The measured distance is displayed on the **Serial Monitor** like:
Distance: 5 cm
- LEDs and servo position change automatically depending on the distance range.





In Serial Monitor



7. Result

After implementing and testing the system, the following observations were recorded:

Close Range (≤ 10 cm):

When an object was detected at a close distance, the green LED turned ON while the yellow and red LEDs remained OFF. The servo motor rotated to 90°, indicating the close-range response. The distance values were continuously displayed on the Serial Monitor, and the output changed quickly as the object moved within this range.

Medium Range (11–20 cm):

When the object was moved slightly farther away, the system switched to the yellow LED, while the green and red LEDs remained OFF. The servo motor moved to 45°, representing the medium-range condition. The transition between close and medium range occurred smoothly as the measured distance crossed the threshold values.

Far Range (> 20 cm):

When the object was positioned beyond 20 cm, the red LED turned ON and the green and yellow LEDs remained OFF. The servo motor returned to 0°, indicating the far-range state. The system maintained this output as long as the object stayed at a distance greater than 20 cm, and it updated automatically when the object distance changed.

8. Conclusion

This project provided valuable hands-on experience in interfacing sensors with a microcontroller and strengthened my understanding of distance-based embedded control systems. By developing the ultrasonic distance measurement setup, I was able to apply programming logic to real hardware components such as the HC-SR04 ultrasonic sensor, LEDs, and a servo motor. The experiment improved my skills in generating trigger pulses, measuring echo duration using **pulseIn()**, converting time into distance values, and

using conditional statements to control outputs based on real-time sensor readings.

It also enhanced my understanding of practical circuit construction, including correct wiring of the sensor, proper use of resistors for LED protection, and maintaining a common ground for stable operation. During implementation, I faced challenges such as fluctuating readings and minor servo jitter, which required checking connections and adjusting timing delays for smoother and more reliable performance. Overall, the experiment successfully linked theoretical knowledge of ultrasonic ranging with practical application and provided a strong base for future work in automation, obstacle detection, and intelligent sensor-based control systems.